

→ Layer 2: Data Link Layer

let's first understand why we need it.

Suppose we want to send a file to your friend.

My Computer ----- Friends computer

The Physical layer (layer 1) can send only

10101010010101000100.

→ It sends only bits.

→ it doesn't know:

- where does the data start?
- where does the data end?
- who is the sender?
- who is the receiver?
- Is the data correct?
- Did any bit change?

Physical layer simply says:

"I only send bits."

So another layer is needed.

that layer is called Data link layer.



→ What is Data Link layer?

The data link layer is the second layer of the OSI model that makes communication **reliable, errorfree and safe** between two directly connected devices/ node-to-node connection.

or,

It prepares the data before it is sent through the physical layer.

→ What does "Data-Link" mean?

Data means:

- File
- Image
- Video
- message

Link:

means  
Connection

Data link means:

**Sending data Correctly** over one direct connection (link).

Real-life eg:

Suppose we want to send a gift.

Without packing:

Gift  
↓  
Courier



The gift may get damaged.

instead,

we,

- Pack the gift
- write sender's name
- write receiver's name
- seal the box

Now its ready.

Similarly,

Data link layer prepares the data before sending.

→ What happens before sending?

Suppose we send

Hello

Computer understands only

101010010001

The data link layer prepares this data:

it :

- Packs it
- adds addresses
- check for errors
- sends it to physical layer.



→ Main function of Data link layer is

There are 5 important functions in

1. Framing
2. ~~Physical addressing~~ Error detection Control
3. Flow control
4. Media Access Control

→ (i) Framing :-

Framing is one of the best most important function of the data link layer.

Framing is the process of ~~dividing~~ dividing a continuous stream of data into small, manageable units called frames before transmission.

In simple words :-

Framing means putting data into a proper frame so that the receiver can identify where the data starts and where it ends.

→ Why do we need framing?

Let's say the sender wants to send:

HELLO



The computer converts it into binary data :

01001000 01000101 01001100 0100100 01001111

Now imagine many messages being sent continuously :

0100100010010101001100.....

The receiver receives a continuous stream of bits.

The problem is :

How does the receiver know which bits belong to which message?

for eg :

|          |          |          |     |
|----------|----------|----------|-----|
| Data A   | Data B   | Data C   |     |
| ↓        | ↓        | ↓        |     |
| 10101010 | 11001100 | 10110101 | ... |

Without some way of marking boundaries, the receiver may not know :

- Where does Data A end ?
- Where does Data B start ?
- Where does Data B end ?

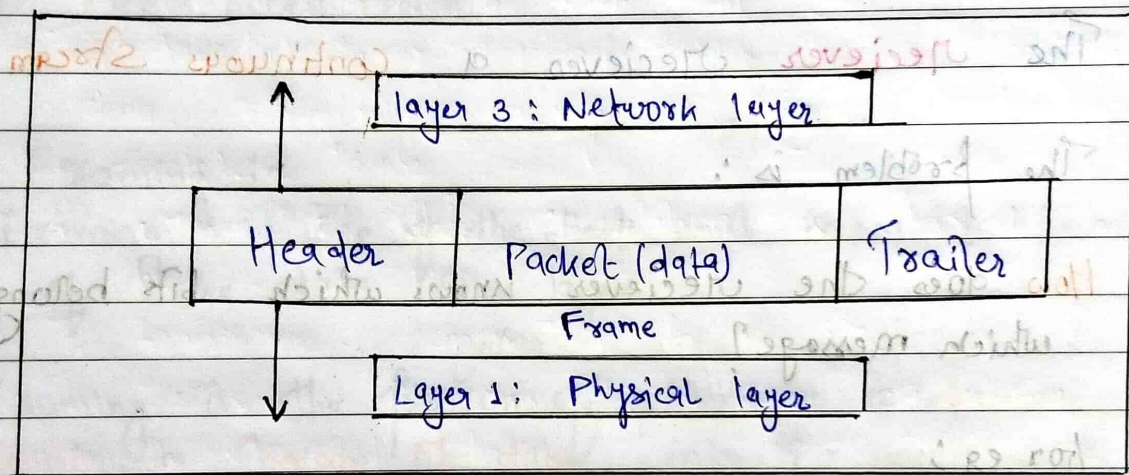
Formatting solve this problem.



→ What is a frame?

⇒ A frame is a ~~unit~~ **unit of data** created by the data link layer.

The data link layer takes the ~~packet~~ **packet** received from the **network layer** and adds information around it.



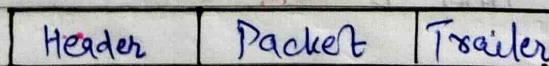
Packet + Data Link layer Header + Data Link layer Trailer = Frame

In simple words;

Suppose the network layer gives the data link layer:

**Packet**

The data link layer adds a header and trailer.



This complete unit is called as: **Frame**



MAC → Media Access control { fifth funcn of data link layer }

→ What does the Header contain?

⇒ The header is the information added before the actual data.

It mainly contains information needed to deliver the frame correctly.

Common information in the header includes:-

- Source MAC address: MAC address of the sender.
- Destination MAC address: MAC address of the receiver.
- Other control information depending on the protocol.

Header

|                     |
|---------------------|
| Source MAC          |
| Destination MAC     |
| Control information |

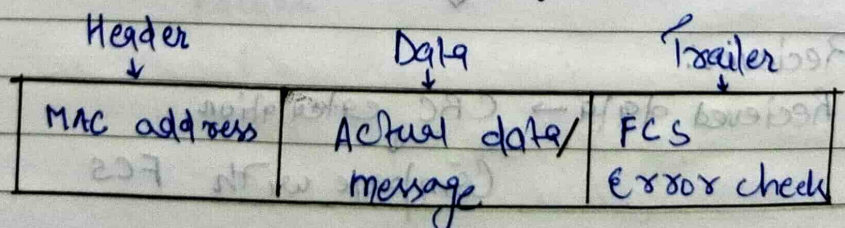
→ What does the trailer contain?

The trailer is the information added after the actual data.

Its main purpose is error detection.

→ The most common example is:

FCS (Frame Check Sequence)





→ FCS :-

Frame check sequence (FCS) is used in the data link layer for error detection.

Its main purpose is to check whether the data in a frame has been corrupted during transmission.

→ How FCS works :-

1. The sender sends the data from the upper layer.
2. The data link layer performs a calculation on the data, commonly using CRC (Cyclic Redundancy Check).
3. The result of this calculation is stored in the FCS field.
4. The frame is transmitted to the receiver.
5. The receiver performs the same calculation on the received data.
6. The receiver compares the calculated value with the received FCS.

Sender:

Data → CRC calculation → FCS

↓

Frame is sent

↓

Receiver:

Received data → CRC calculation

↓  
Compare with FCS



- FCS matches:~ No transmission error is detected.
- FCS does not match:~ An **error** is detected in the frame.

Note:~

FCS detects errors but does not correct them.

→ Complete framing process:~

Suppose you want to send a message.

"Hello"

Step 1: Application ~~layer~~ creates data

Hello

Step 2: Higher layers process it

Eventually, the network layer creates a:

Packet

Step 3: Data Link layer receives the packet

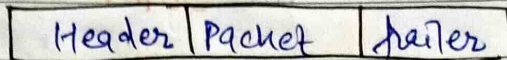
Packet

↓

Data Link layer



Step 4: Data link adds a header and a trailer



Now we have a:

**Frame**

Step 5: Frame goes to physical layer

The physical layer converts the frame into **bits/signal** and transmits them.

Frame



Bits



Signal



Transmission medium

→ Main purpose of framing is:

Framing mainly helps the receiver in identifying the **boundaries of data**.

That means:

Start of frame



|            |
|------------|
| Frame Data |
|------------|



End of frame



(ii) Error Control :-  
 Error control in the data link layer ensure that frames transmitted by the sender are received correctly and reliable by the receiver.

In other words:

Error control is the process of detecting and handling errors that occur during data transmission.

→ Why do we need error control?

Suppose a computer sends:

1011010

Because of interference, Hardware problem, Electrical disturbance and weak signal

1010010

The data has changed

Sender

Receiver

1011010

→

1010010

↑  
Error

If nothing is done, the receiver may use wrong data.

∴ we need Error Control



→ What does control do?

Error control mainly involves two activities:

1. Error detection:-

find out:

"Is there an error".

2. Error correction:-

Take appropriate action:

"How should the error be corrected or recovered from?"

### Error Control

Error detection

↓  
find the error

Error correction

↓  
fix/recovered data

→ What is Error detection?

Error detection is the process of determining whether the received data contains an error.

In simple words;

Error detection tells us whether the data is correct or corrupted.



→ Techniques of error detection in data link layer :-  
 Error detection means finding whether the data received by the receiver is **correct** or **corrupted** during transmission.

The main techniques are :-

1) Parity check :-

A parity bit is added to the original data to make the number of 1s either **even** or **odd**.

There are two types :

- Even parity: Total no. of 1s should be even.
- Odd parity: Total no. of 1s should be odd.

Eg:

Data = 10 11001

No. of 1s = 4

for even parity → parity bit = 0

Transmitted data: 10110010

If the receiver finds an incorrect number of 1s, it detects an error.

Limitation: It cannot **reliably** detect errors when an **even number of bits** are **changed**.



2. Checksum: In the checksum method, data is divided into fixed-size blocks.

Steps:

1. Divide data into equal-sized blocks.
2. Add all the blocks using 1's Complement addition.
3. Take the 1's complement of the sum.
4. The result is called the checksum.
5. Receiver performs the same calculation to detect errors.

Eg:

Let's take two 4-bit data blocks:

Block 1 = 1010

Block 2 = 1100

Step 1:

Add the blocks:

$$\begin{array}{r} 1010 \\ 1100 \\ \hline 10110 \end{array}$$

We get 5 bit but our block is 4 bit

Step 2: Add the carry back:

$$\begin{array}{r} 0110 \\ + \quad 1 \\ \hline 0111 \end{array}$$



So, sum = 0111

Step 3:-

Take 1's complement:

Sum = 0111

1's comp. = 1000

Checksum = 1000

Step 4: ~ What is transmitted?

The sender transmits:

1010

1100

1000

Block 1

Block 2

~~Block 3~~

Checksum

At receiver :-

Receiver adds all three:

1010

+ 1100

+ 1000

10010

add the carry back

0010

+ 1

0011



then take 2's complement:

$$0011 \rightarrow 1100$$

This result is not matched:

Checksum = 1000

but here, 1100

Error found.

Eg:

Block 1: 1010

Block 2: 0101

Addition:

$$\begin{array}{r} 1010 \\ + 0101 \\ \hline 1111 \end{array}$$

1's complement

$$1111 \rightarrow 0000$$

Checksum = 0000

Receiver:

$$\begin{array}{r} 1010 \\ 0101 \\ 0000 \\ \hline 1111 \end{array}$$



1's complement :

1111  $\rightarrow$  0000 //

Since, final result is 0000, the data is considered error-free.

3. Cyclic Redundancy Check (CRC) :-

CRC is a powerful and widely used error-detection technique.

It uses binary division.

Steps :-

1. Take the original data
2. Append zeros to the data.
3. Divide it by a predetermined generator polynomial using modulo-2 division.
4. The remainder is called the CRC
5. Append the CRC to the original data.
6. Receiver divides the received frame by the same generator.
7. If the remainder is 0, the frame is assumed to be error-free.
8. If the remainder is non-zero, an error is detected.

here,

Append zero formula:

$n-1$

where,

$n$  is no. of bit of divisor



→ modulo-2 division :-

Subtraction = XOR

$$0 \oplus 0 = 0$$

$$0 \oplus 1 = 1$$

$$1 \oplus 0 = 1$$

$$1 \oplus 1 = 0$$

and:

- If the first bit is 1 → XOR with divisor
- If the first bit is 0 → don't XOR, move to next bit
- Final **Remainder** has  $(n-1)$  bits, where  
 $n$  = no. of bits of divisor bits.

Eg ① Divide:

$$1101 \div 101$$

Divisor = 101 → 3 bits

So remainder will be **2 bits**  $[(n-1)]$

Step 1:-

Take first three bits

$$\begin{array}{r} 110 \\ 101 \\ \hline 011 \end{array}$$

first is now 0, so move **1** right and bring down 1



$$011 \rightarrow 111$$

Step 2:-

$$\begin{array}{r} 111 \\ 101 \\ \hline 010 \end{array}$$

No bits left:

$$\therefore \text{Remainder} = 10$$

$$1101 \div 101 = \text{Remainder } 10$$

Eg ②

$$1001 \div 11$$

Divisor  $\rightarrow 11 \rightarrow 2$  bits

Remainder  $\rightarrow 1$  bit

Step 1:-

$$10$$

$$\begin{array}{r} 10 \\ 11 \\ \hline 01 \end{array}$$

Bring down 0

Step 2:-

$$\begin{array}{r} 10 \\ 11 \\ \hline 01 \end{array}$$

Bring down 1



Step 3:-

$$\begin{array}{r} 11 \\ 11 \\ \hline 00 \end{array}$$

$\therefore$  remainder  $\rightarrow 0$  [1 bit]

Eg:  $00100 \div 1011$

divisor  $\rightarrow 4$  bit  
remainder 3 bit

here, we can shift right until 1 come ~~here~~  
~~we only shift two zeros. Because we need 4~~  
~~bit because divisor is 4 bit~~

~~$$\begin{array}{r} 0101 \div 1011 \\ 0101 \\ \hline 1011 \\ \hline 1110 \end{array}$$~~

$$100 \div 1011$$

There are no bit to bring down

$\therefore$  final remainder  $\rightarrow 100$



PYQ - 2024

Q. Given a 10 bit sequence 1010011110 and a divisor. Find the CRC - check your answer.

⇒

Given;

data = 1010011110

divisor = 1011

divisor has 4 bits,  
so append

$$4 - 1 = 3 \text{ zeros}$$

∴

original data = 1010011110

Append = 000

Divident = 1010011110000

Now perform modulo-2 division

Step 1:-

$$\begin{array}{r} 1010 \\ 1011 \\ \hline 0001 \end{array}$$

Step 2:

first bit is zero so bring down 0

0010

again, first ~~second~~ bit is zero so bring down 1

0101

again, first bit is zero so bring down 1

1011



Step 3:-

$$\begin{array}{r} 1011 \\ 1011 \\ \hline 0000 \end{array}$$

Step 4:-

bring down 1

$$0001$$

again, bring down 1

$$0011$$

again, bring down 0

$$0110$$

again, bring down 0

$$1100$$

Step 5:-

$$\begin{array}{r} 1100 \\ 1011 \\ \hline 0111 \end{array}$$

Step 5:-

bring down 0

$$1110$$



Step 6:-

$$\begin{array}{r} 1110 \\ 1011 \\ \hline 0101 \end{array}$$

Step 7:-

bring down the final bit 0.

$$1010$$

Step 8:-

$$\begin{array}{r} 1010 \\ 1011 \\ \hline 0001 \end{array}$$

There are no more bit to bring down

Since the divisor has 4 bit, the remainder is the last 3 bits

$$\text{remainder} = 001$$

$$\boxed{\text{CRC} = 001}$$

Now,

final transmitted code:

original data + CRC

$$1010011110 + 001$$

$$= 1010011110001$$



→ At the receiver:

the receiver receives:

101001110001

→ The receiver divides the complete received codeword by the same divisor.

$$101001110001 \div 1011$$

here,

If remainder = 000 → No errors detected / Error-free

If remainder  $\neq$  000 Error detected

→ What is Error correction?

Error correction is the process of recovering the correct data after an error has been detected.

Error correction is mainly done in two ways:

1. Forward Error (FEC)

2. Retransmission / ARQ (Automatic Repeat Request)

1. Forward error Correction (FEC):

In FEC, the sender adds extra / redundant bits to the original data.



The receiver uses these extra bit to:

- Detect the error
- Find the position of the error
- Correct the error without asking the sender to transmit the data.

Eg;

Suppose the sender sends:

Data + redundant bits

10110101...

If one bit gets changed during transmission:

Sent: 10110101

Received: 10100101

↑

Using the **redundant bits**, the receiver can determine the position of the **wrong bit** and correct it.

Common technique:

**Hamming code** is a common example of FEC.

2.4 Hamming Code:

Hamming code is used to **detect and correct errors** in transmitted data.

It adds **parity bits** at specific positions.



Eg;

Positions :

|               |               |               |               |               |               |               |
|---------------|---------------|---------------|---------------|---------------|---------------|---------------|
| 1             | 2             | 3             | 4             | 5             | 6             | 7             |
| <del>P1</del> | <del>P2</del> | <del>D1</del> | <del>P4</del> | <del>D2</del> | <del>D3</del> | <del>D4</del> |
| P1            | P2            | D1            | P4            | D2            | D3            | D4            |

Here,

- D = Data bits

- P = Parity bits

The parity bits helps the receiver identify the **exact position of an erroneous bit.**

Eg;

Sent: 1011001

Received: 1010001

↑  
Error

Hamming code calculates a value called the **syndrome**, which indicates the position of the errors.

The receiver can flip that bit:

Wrong bit: 0

Correct: 1

Thus, the original data is recovered.



### 3. Automatic Repeat Request (ARQ) :-

In ARQ, the receiver does **not directly** the **corrupted frame**.

Instead:

1. Receiver detects an error.
2. Receiver asks the sender to send the frame again.
3. Sender retransmits the frame.



Flow

(iii) ~~Flow~~ Control :- (R/A)   
 Flow Control is a function of the data link layer that control the rate at which the sender sends data so that the receiver can receive and process it properly.

In simple words:

**Flow Control:** Controlling the speed of data transmission according to the receiver's capacity.

→ Why do we need flow control :-

Suppose there are two computers :-

Sender



Fast

Receiver



Slow

→ Suppose the sender can send:

1000 frames/second

→ But the receiver can process only:

100 frames/second

• The sender is sending data much faster than the receiver can process it.



Sender  
 1000 frames/sec



Receiver  
 100 frame/sec

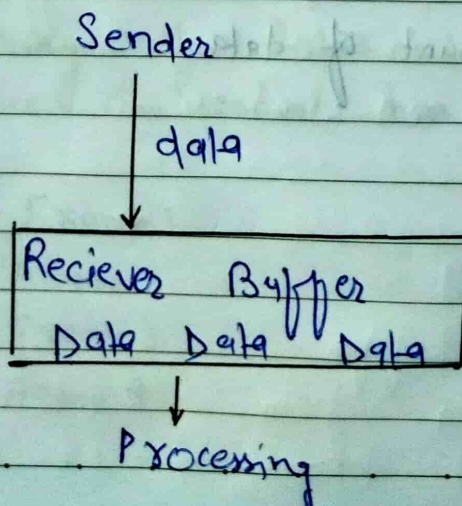
What can happen?

- Receiver's buffer can become full.
- Frames may be lost.
- Data may need to be retransmitted.
- Network performance can decrease.

Therefore, we need **flow control**.

→ What is a Receiver Buffer?  
 Before understanding flow-control methods, understand the **buffer**.

A buffer is a temporary memory area used to store data while it is waiting to be passed.





Suppose the receiver has space for 5 frames.

Buffer:

[Frame 1]  
[Frame 2]  
[Frame 3]  
[Frame 4]  
[Frame 5]

If the sender keeps sending more frames, the buffer may become full.

Buffer FULL

If more data arrives, it may be discarded.

Flow control helps prevent this situation.

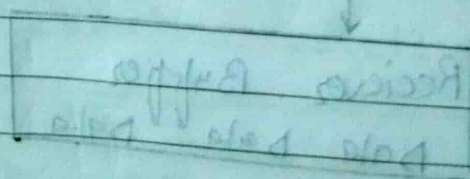
→ Main idea of flow control is

Fast sender



Controlled amount of data

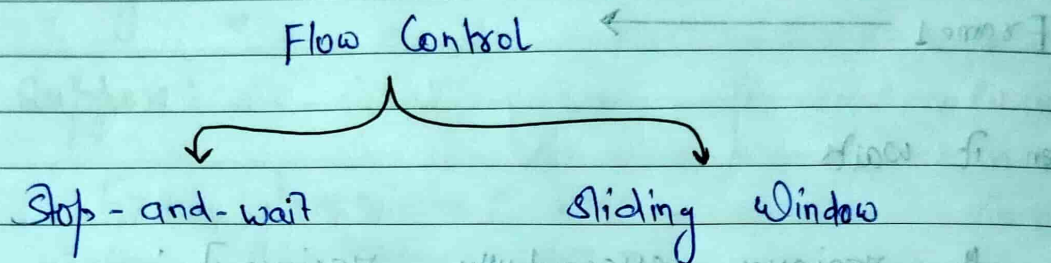
Slow Receiver





→ Types of flow control :-

The two important basic flow-control techniques are:



(i) Stop-and-wait flow control :-

In Stop and wait, the sender sends **one frame** **at a time** and **wait** for the receiver to indicate that it can **continue** / has received the frame.

Send one frame

↓  
wait

↓  
Continue

↓  
Send next frame

→ Working of stop-and-wait  
 Suppose the sender has four frames:

Frame 1

Frame 2

Frame 3

Frame 4



ACK → Acknowledgement

ACK is a message sent by the receiver to tell the sender that the data was received successfully.

The sender send frame 1::

Sender Receiver

Frame 1 →

Then it waits

After the receiver successfully receive it :

Sender

Receiver

Frame 1 →

ACK ←

So on...

→ Why does stop-and-wait provide flow control?  
Because the sender can have only a limited amount of unacknowledged data in transit.

Therefore, a fast sender cannot continuously flood the receiver with frames.

→ Advantages of Stop-and-wait :-

- Very simple
- Easy to implement
- Easy to understand
- Requires a small buffer
- Prevents the sender from overwhelming the receiver.



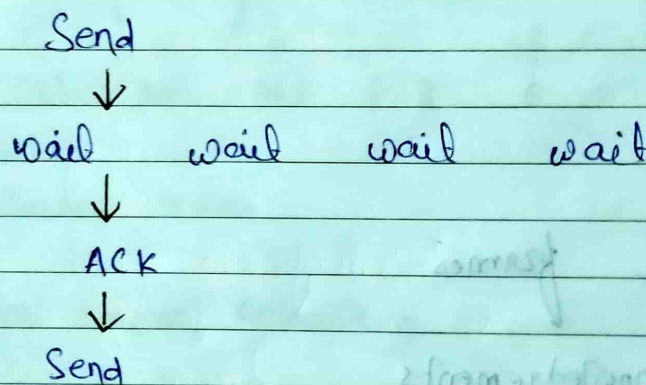
→ Disadvantages :-

The major problem is **low efficiency**, especially when the communication link has a large delay.

Suppose:

Frame transmission = 1 ms  
 waiting for response = 100 ms

The sender spends most of its time waiting



This wastes available network capacity.

∴, a faster method is needed.

That method is

**Sliding window**

(ii) Sliding Window flow Control :-

Sliding window allows the sender to send **multiple frames before waiting for acknowledgement**.



Instead of:

Frame 1

wait

Frame 2

wait

Frame 3

wait

we can have:

Frame 1

Frame 2

Frame 3

Frame 4



Send several frames



Receive acknowledgements

This makes communication much more efficient.

→ Basic idea:

Suppose the window size is 4.

The sender can send four frames without waiting for individual acknowledgements.

Window Size = 4

[Frame 1] [Frame 2] [Frame 3] [Frame 4]



After acknowledgement are received, the window moves forward.

Eg:

Initially:

[1] [2] [3] [4] 5 6 7 8  
 ↑  
 Window

After frame 1 is acknowledged:

1 [2] [3] [4] [5] 6 7 8  
 ↑  
 Window moves

After more acknowledgments

1 2 [3] [4] [5] [6] 7 8  
 ↑  
 window

That's why it is called **Sliding window**

→ Why Sliding window is better?

Suppose:

Window size = 4

Sender can send:



### Frage 4

1. 2. 3. 4. 5. 6. 7. 8. 9. 10. 11. 12. 13. 14. 15. 16. 17. 18. 19. 20. 21. 22. 23. 24. 25. 26. 27. 28. 29. 30. 31. 32. 33. 34. 35. 36. 37. 38. 39. 40. 41. 42. 43. 44. 45. 46. 47. 48. 49. 50. 51. 52. 53. 54. 55. 56. 57. 58. 59. 60. 61. 62. 63. 64. 65. 66. 67. 68. 69. 70. 71. 72. 73. 74. 75. 76. 77. 78. 79. 80. 81. 82. 83. 84. 85. 86. 87. 88. 89. 90. 91. 92. 93. 94. 95. 96. 97. 98. 99. 100.

- 10/10/2020

8 15 20 25 30 35 40 45 50 55 60 65 70 75 80 85 90 95 100

8 7 (2) (1) (P) (E) S

combined with balloons as Fi put a i fast

filled a notebook with 2 pages a



(iv) Media Access Control (MAC) is an important function of Data link layer (layer 2) of the OSI model.

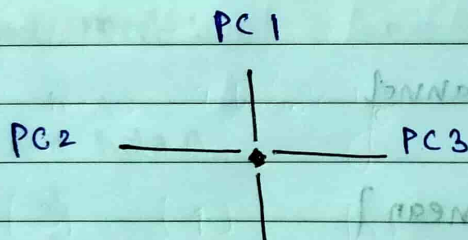
Media Access Control is the method used to control which device can access and use a shared communication medium to transmit data.

In simple words

MAC decides "who can send data and when"?

→ Why do we need Media Access Control?

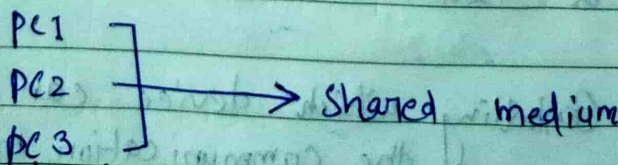
Suppose four computers are connected to the same communication medium:



Now suppose:

- PC1 wants to send data.
- PC2 also wants to send data.
- PC3 also wants to send data.

If everyone sends at the same time:





Their signals may interfere with each other.

This can cause a:

**Collision**

Therefore, we need a mechanism to decide:

**Which device gets to transmit!**

This is the job of **Media Access Control**.

→ What does "Media" mean?

Here, media means **communication medium** through which data travels.

Eg: Ethernet cables

• Coaxial cables

• Wireless radio channel

→ What does "Access" mean?

Access means the **right or opportunity** to use something.

So:

Media Access = Getting permission/opportunity to use the communication medium.

→ What does "Control" mean?

Control means managing or deciding how something is used.

∴

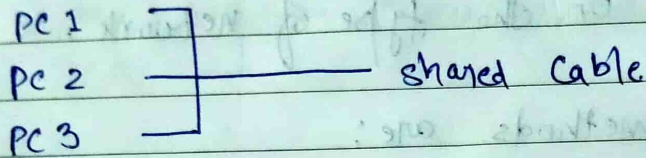
Media Access Control = Controlling which device can be used the communication medium and when.



→ Media Access Control and Collision:~

let's us understand:

Suppose:



PC1 start transmitting:

PC1 → Data

same

At the time, PC2 also start transmitting:

PC2 → Data

The signal meet:

Data A + Data B

↓

Collision

The data may become corrupted.

MAC mechanism are used to coordinate access to the medium and, in some technology, detect or avoid collisions.



→ How does MAC control Access?  
Different networking technologies use different **MAC protocols**.

The method depends on the type of network.

Some important methods are:

1. CSMA/CD

2. CSMA/CA

~~CSMA/CD~~

→ (i) CSMA/CD :-

CSMA/CD stands for:

Carrier Sense Multiple Access with Collision Detection.

It was used in traditional **shared, half-duplex Ethernet**.

~~Let's break it down~~

• Carrier Sense:

Before sending, the device checks:  
"Is the communication medium free?"

Medium free?

↓

Yes

↓

Transmit

No

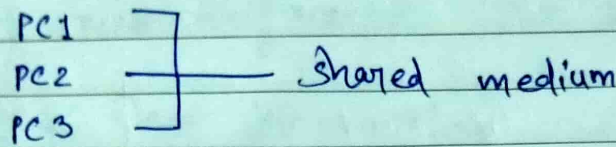
↓

Wait



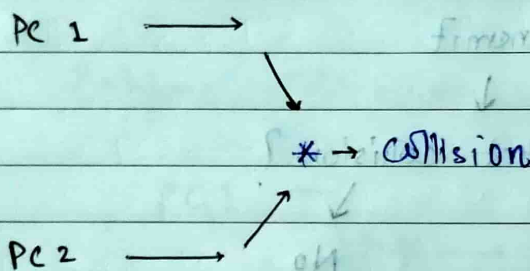
### Multiple Access:

Many devices share the same communication medium.



### Collision Detection:

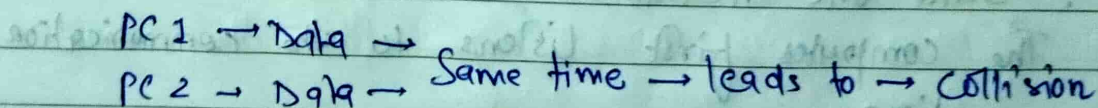
If two devices transmit at the same time, they may detect a collision.



The device detect the collision and stop transmitting

→ Why is CSMA/CD Needed?

If two computer transmit simultaneously on a shared network.

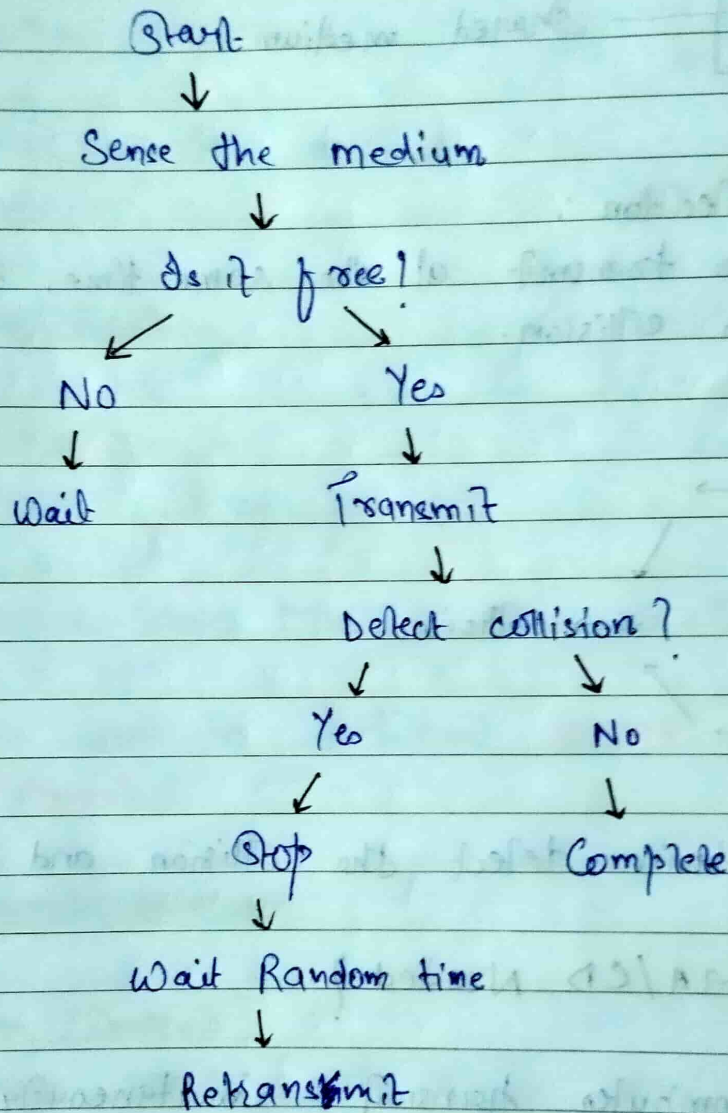


CSMA/CD provides a way to detect and handle such collision.



## → Working of CSMA/CD :-

The basic process is :



Step 1: Sense the medium:

The computer first listens to the communication medium.

PC1 → "Is anyone transmitting?"

If another device is transmitting:  
Medium = Busy

PC1 waits



Step 2: Medium is free:

If the medium is free

medium = free

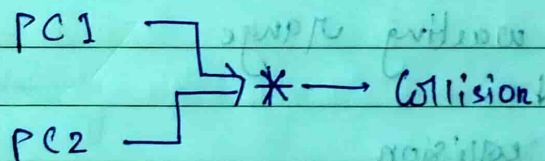
PC1 Start transmitting

PC1 → Data →

Step 3: Detect collision:

While transmitting, the device continues to monitor the medium.

Suppose PC2 also starts transmitting:



A collision occurs.

Step 4: Stop transmission:

Both devices stop transmitting.

PC1 → Stop

PC2 → Stop

Step 5: Wait for a Random time:

The device do not immediately transmit again. They wait for a random amount of time. Because if both waited for exactly the same amount of time, they might transmit together again.



### Step 6: Retransmit:

After waiting; the devices try again.

PC1 — Data → transmitted successfully

### → Binary Exponential Backoff:

The random waiting process ~~in collision~~ is called **Binary Exponential Backoff**.

After repeated collisions, the range from which the waiting time is selected becomes larger.

1<sup>st</sup> collision



Small waiting range



2<sup>nd</sup> collision



large waiting range



3<sup>rd</sup> collision



Even larger range

This reduces the probability that the same device will collide again.



→ Advantages :

- Detect collision
- Reduces repeated collision using backoff
- efficient when network traffic is relatively low.

→ Disadvantages :-

- Collision <sup>can</sup> occur.
- Performance decreases as network traffic increases.
- Time is wasted when collisions occurs.

(ii) CSMA/CA :-  
It stands for:

CSMA/CA = Carrier Sense Multiple Access with Collision Avoidance.

It is primarily used in wireless LANs (Wi-Fi).

CSMA/CA is a method in which wireless device listens to the channel and uses waiting/backoff and other mechanism to reduce the probability of collisions before transmitting.

→ Why CSMA/CA Needed?

Wireless communication is different from wired communication.

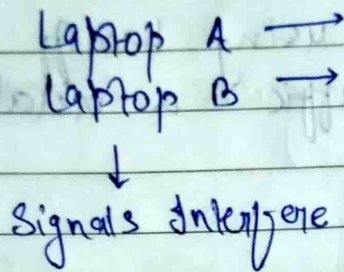
Consider:

Laptop A >>>>> Access point  
Laptop B >>>>> Access point

Both laptops use the same radio channel.



7) both transmit at the same time:



A collision can occur.

But a wireless device generally cannot reliably listen for a collision while transmitting in the same way that CSMA/CD could.

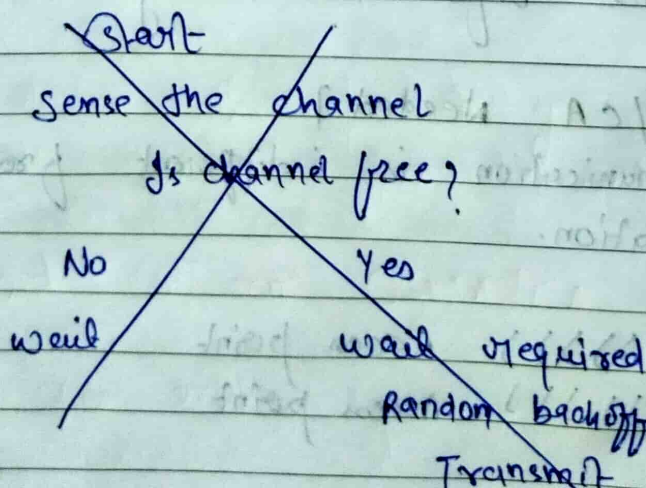
Wi-Fi mainly tries to:

Avoid collisions before transmission.

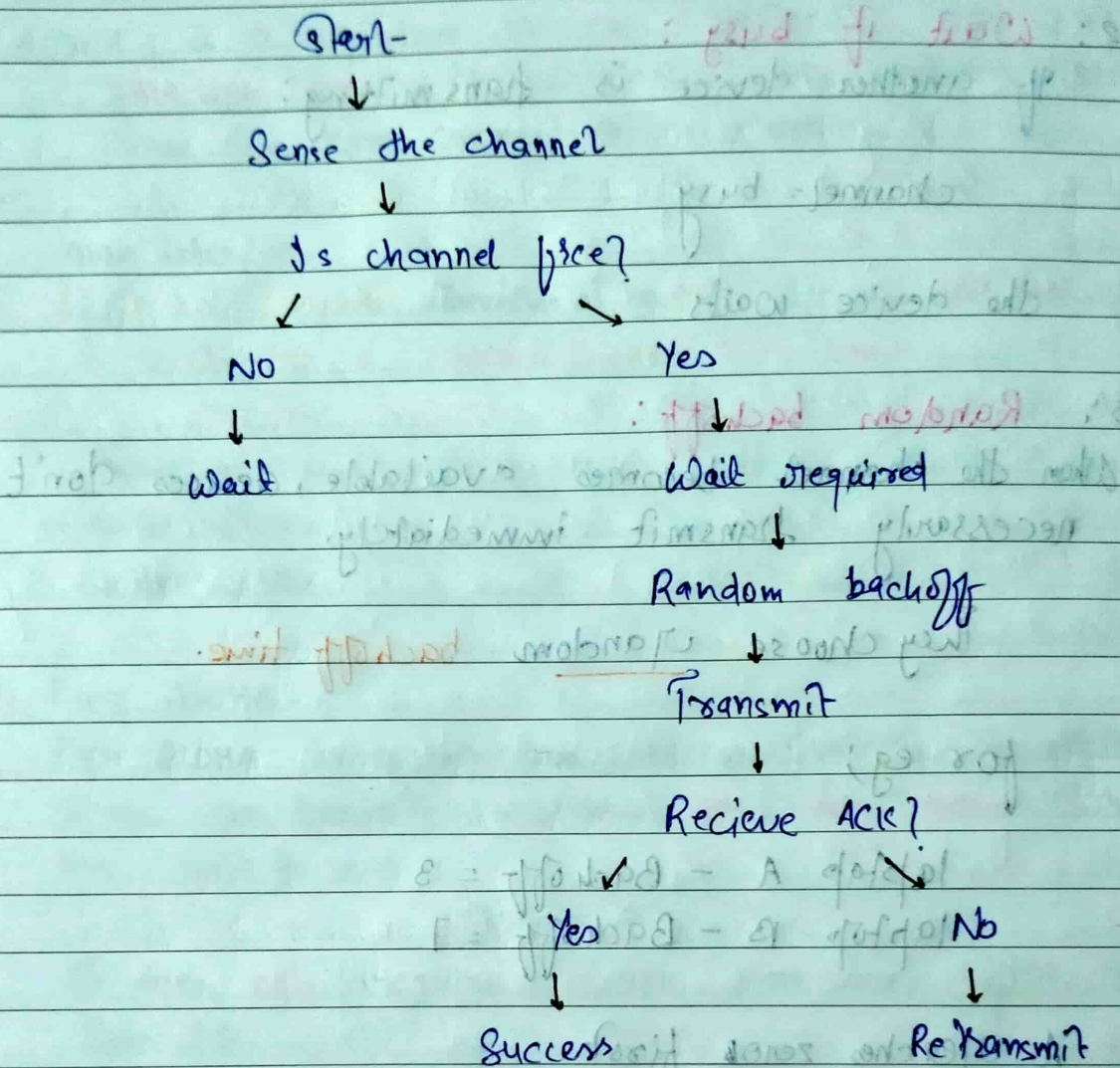
That's why we have

CA = Collision Avoidance.

→ Basic Working:







Suppose two laptops want to send data over wifi:

Laptop A

Laptop B

11 > > > > wifi > > > >

Step 1: Sense the channel:

Both devices listen

A → channel free?

B → channel free?



Step 2: Wait if busy:  
If another device is transmitting:

channel = busy

the device waits

Step 3: Random backoff:

When the channel becomes available, devices don't necessarily transmit immediately.

They choose random backoff time.

for eg;

laptop A - Backoff = 3

laptop B - Backoff = 7

A reaches zero first

A → transmit

B waits

This reduces the probability that both devices will transmit simultaneously.

To avoid collision CSMA/CA uses ACK.

Wifi uses acknowledgement at the MAC layer.



→ ALOHA:~

ALOHA is a random access protocol used at the (MAC sub) Data Link Layer to control how multiple devices share the same communication channel.

main idea:~

A device transmits whenever it has data. If a collision occurs, it waits for a random time and tries again.

There are two main types:

1. Pure ALOHA

2. Slotted ALOHA

1. Pure ALOHA:

Pure ALOHA is a random access protocol in which a device can transmit a frame at any time whenever it has data to send.

It does not require device to wait for a fixed time slot.

In simple words,

If a device has data:

Send it immediately

If two devices transmit at nearly the same time, their frames may collide.

→ Working of pure ALOHA:~

Suppose there are three devices:

Shared Communication channel

PC1

PC2

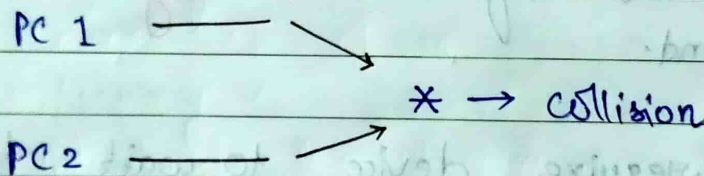
PC3



Steps:

1. A device gets a frame to send.
2. It immediately transmits the frame.
3. If no other device transmits at the same time, the frame is successfully received.
4. If another device transmits at the same time, a collision occurs.
5. The sender waits for a random amount of time.
6. It retransmits the frame.
7. This process continues until successful transmission or until the protocol gives up after its retry limit.

If two devices transmit together:



After collision:

PC 1 → Random wait → Retransmit

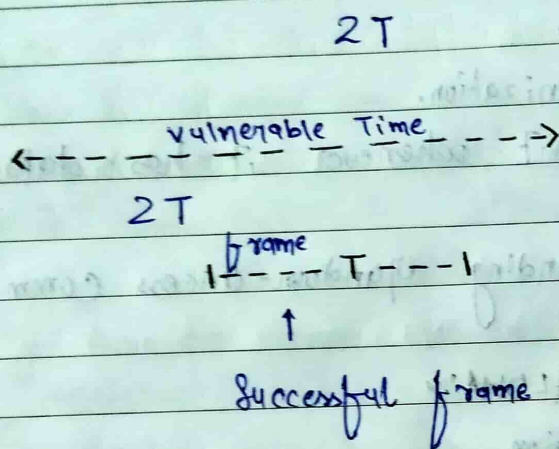
PC 2 → Random wait → Retransmit



→ Vulnerable Time in Pure ALOHA :-  
Suppose one frame take  $T$  seconds to transmit.

For the frame to be successful, another frame must not start transmission during a period of  $2T$ .

vulnerable Time :



A collision can occur if another frame starts:

- during the frame's transmission, or
- during the  $T$  time just before its transmission start.

Efficiency of Pure ALOHA :-

The max theoretical throughput of Pure ALOHA is :-

$$S = G e^{-2G}$$

where :

- $S$  = throughput
- $G$  = offered traffic load
- $e$  = euler's number  $\approx 2.718$



Max<sup>m</sup> throughput is:

$$18.4\% \approx 0.184$$

Pure ALOHA can use the channel efficiently only up to about 18.4% max<sup>m</sup> theoretical throughput.

Advantages of Pure ALOHA:-

1. Very simple protocol.
2. No need for synchronization.
3. A device can transmit whenever it has data.
4. Easy to implement.
5. Useful for understanding random-access communication.

Disadvantages of Pure ALOHA:-

1. High chance of collision.
2. Low efficiency.
3. Max<sup>m</sup> theoretical throughput is only 18.4%.
4. Retransmission waste bandwidth.
5. Performance become poor when many devices are transmitting.

2. Slotted ALOHA:

Slotted ALOHA is an improved version of pure ALOHA in which time is divided into equal-size slots.

A device can transmit only at the beginning of a time slot.

In simple words:

Slotted ALOHA:

'Send only when a slot starts'.



→ Working of Slotted ALOHA :-  
 Suppose the communication channel is divided into slots.

Time →  
 | s<sub>1</sub> | s<sub>2</sub> | s<sub>3</sub> | s<sub>4</sub> | s<sub>5</sub> |

Each slot is large enough to transmit one complete frame.

Steps :-

1. Time is divided into equal slots.
2. A device waits until the beginning of a slot.
3. If it has data, it transmits during that slot.
4. If only one device transmits, transmission is successful.
5. If two or more devices transmit in the same slot, a collision occurs.
6. The device waits for a random future slot and retransmits.

Eg:

Suppose three devices have data.

Slot → 1 2 3 4 5

PC1 Data

PC2 Data

PC3 Data

here, only one device transmits in each slot.  
 therefore,

Slot 1 → PC1 → Successful

Slot 3 → PC2 → Successful

Slot 4 → PC3 → Successful

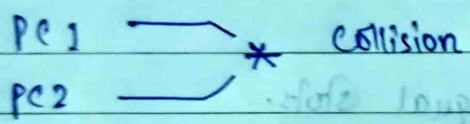


But;

slot

PC1 Data  
 PC2 Data

Both PC1 and PC2 transmit in slot 1.



They wait for random future slots and try again.

→ Vulnerable time in Slotted ALOHA:  
 for slotted ALOHA, the vulnerable period is :

where,

$T$  = one frame transmission time.

This is smaller than pure ALOHA's vulnerable period.

Pure ALOHA →  $2T$   
 Slotted ALOHA →  $T$

Because transmission can begin only at slot boundaries, the chance of collision is reduced.



→ Efficiency of slotted ALOHA is

The throughput is :

$$S = G e^{-G}$$

max<sup>m</sup> theoretical throughput is :

$$36.8\% \approx 0.368$$

So, slotted ALOHA is approx. twice as efficient as pure ALOHA.

→ Why is Slotted ALOHA better?

The main problem in pure ALOHA is that a device can start transmission at any moment.

for eg;

PC 1 : 1---Data----

PC 2 : 1---Data----



collision

In slotted ALOHA, transmission starts only at slot boundaries.

slot 1

slot 2

slot 3

PC 1 : 1-Data-1

PC 2 : 1-Data-1

If two device transmit in the same slot, there is still a collision, but the vulnerable period is reduced from  $2T$  to  $T$ .

Akash